RAG Assignment: Baseline vs. Re-Ranking

Karthik Nallala
Roll No. 2025201095

Code: `https://github.com/karthik443/RAG_Project`

I used the CATAN rulebook and almanac (16 pages) as the document. It is split into 82 chunks of roughly 110 words. The bi-encoder is `all-MiniLM-L6-v2` with a FAISS inner-product index, the re-ranker is `ms-marco-MiniLM-L6-v2`, and the LLM is Qwen2.5-3B-Instruct running locally on CPU. Both pipelines use the same chunks, same top-10 and same prompt, so the only difference is which three chunks go to the LLM.

1 Method

The PDF text was read with PyMuPDF and cleaned (copyright lines and the dice-dot symbols removed). Blocks from the same page were joined into chunks of about 130 words or fewer, with no overlap. The chunk embeddings are normalised, so the FAISS inner product equals cosine similarity.

Pipeline 1 takes the 10 nearest chunks and sends the first three to the LLM. Pipeline 2 scores the same 10 chunks with the cross-encoder, sorts them by that score and sends the new top 3. The prompt tells the model to answer only from the given passages, to say "the context does not say" if the answer is missing, and to keep the answer to three sentences. Decoding is greedy, so any difference between Answer 1 and Answer 2 comes from the retrieved chunks and not from sampling.

The queries were picked by testing many questions against the baseline and keeping the ones where the key chunk was not in its top 3, one or two per failure type from the assignment. I kept two cases where re-ranking did not help (Q3, Q9) and an easy control (Q10). I identified the key chunk for each query by hand, using a text pattern, and only to report ranks; the pipelines never see it.

2 Bi-Encoder vs. Cross-Encoder

A bi-encoder turns the query and every chunk into separate vectors and compares them with cosine similarity. The chunk vectors do not depend on the query, so they are computed once and stored, and search is cheap. The downside is that a whole chunk is squeezed into one vector without knowing the question, so words like "not", "only" or "except" are easily lost.

A cross-encoder reads the query and the chunk together as a single input, so self-attention can relate every query token to every chunk token, and it outputs a relevance score directly. It is more precise because of that, but nothing can be precomputed: every query–chunk pair needs its own forward pass.

3 Rank Shifts and Answers

I ran ten queries. Table 1 shows the position of the chunk that actually contains the deciding clause, in the baseline top-10 and after re-ranking. Only ranks 1–3 reach the LLM.

	Query	Type	Before	After
Q1	Bought a VP card that gives 10 points, must I wait?	Exception	7	3
Q2	Can I play a progress card again later?	Exception	10	2
Q3	Resources if a robber-blocked number is rolled?	Exception	7	4
Q4	Can I play two knight cards in one turn?	Negation	6	1
Q5	Can I build a sixth settlement?	Negation	7	2
Q6	More than one road on the same path?	Negation	8	2
Q7	Which hex never produces anything?	Keyword trap	4	1
Q8	Who gets starting resources, and from where?	Buried chunk	10	1
Q9	Do roads count toward my score?	Limitation	>10	>10
Q10	Can a player win on an opponent's turn?	Control	1	1

Table 1: Rank of the answer chunk before and after re-ranking.

For seven of the nine failure queries (Q1, Q2, Q4–Q8) the key chunk was moved into the top 3. The largest moves were Q2 and Q8 (10 to 2, and 10 to 1).

The baseline answers went wrong in a few places. In Q8 the LLM described normal dice-roll production instead of the starting resources, because the setup chunk was at rank 10. In Q7 it said the hex marked "7" produces nothing, which is wrong as there is no 7 token; after re-ranking it said the desert. In Q1 the baseline only repeated the general "10 points on your turn" rule and left out the exception that a just-bought victory point card can be revealed at once, which Answer 2 stated. In Q2 and Q5 both answers were "no", but Answer 1 gave a wrong reason (the one-card-per-turn rule in Q2, a confused explanation of the settlement supply in Q5) while Answer 2 gave the right one. In Q4 and Q6 the baseline answers were already correct, so the improvement there is only in the ranks.

Table 2 puts the two answers side by side for three of these queries (shortened).

	Answer 1 (baseline top-3)	Answer 2 (re-ranked top-3)
Q7	"The hex marked with 7 never produces anything... there is no 7 number token."	"The desert hex never produces anything for anybody."
Q8	Describes players with a settlement next to a forest or hills hex getting brick or lumber (normal dice production).	"Players receive their starting resources immediately after building their second settlement... one card for each adjacent hex."
Q1	"If you have or reach 10 victory points on your turn, the game ends immediately." (general rule only)	"If you buy a victory point card that brings you to 10 points... you may immediately reveal this card and win."

Table 2: Baseline and re-ranked answers for three queries.

Two cases did not work. In Q3 re-ranking moved the chunk from 7 to 4, still outside the top 3, so nothing changed. In Q9 the chunk saying roads only give points through Longest Road was not in the baseline top-10, so re-ranking could not find it and both answers wrongly said roads count. Re-ranking can only reorder what the first stage returns. Also, a good rank does not guarantee a good answer: in Q2 the 3B model quoted an unrelated passage from the right chunks.

4 Latency

These are CPU timings (no GPU), averaged over 20 runs for each of the ten queries.

Stage	Mean time
Baseline: query embedding + FAISS top-10	9.2 ms
Extra for re-ranking: cross-encoder on 10 pairs	276 ms
Re-ranking pipeline, total retrieval	285 ms
LLM generation, Answer 1 / Answer 2 (3 queries)	$\approx$ 99 s / 109 s

Retrieval gets about 30 times slower with re-ranking, but it is still small compared to generating the answer. The LLM numbers are rough, since the model runs on a shared CPU.

5 Why Not Run the Cross-Encoder on Everything?

Its cost grows linearly with the number of chunks, and none of it can be cached because each score depends on the query. I measured about 52 ms per pair here, which is roughly 52 seconds per query for 1,000 chunks and around 14 hours for a million. The bi-encoder only has to embed the query at search time, and a flat FAISS search over a million vectors took about 113 ms. So the cheap model narrows the collection to a few candidates, and the expensive one is only used on those.

6 Conclusion

On this rulebook the cross-encoder was most useful for questions about exceptions, limits and prohibitions, where the deciding sentence sits in a chunk that is only loosely similar to the question. The first stage usually had that chunk somewhere in its top 10, and the cross-encoder was good at moving it up. The cost is about 0.3 seconds per query on CPU, which is small next to the LLM.

Some limits are worth stating. The test set is only ten queries on one document, so these are examples rather than a statistical result. The cross-encoder cannot fix a miss in the first stage (Q9), and it did not always get the chunk into the top 3 (Q3). The 3B model is also not a very strong reader, so it sometimes misused good context. A larger LLM, a larger candidate pool (top-20 instead of top-10) or chunks that keep an almanac entry together would probably help, but I did not test these.